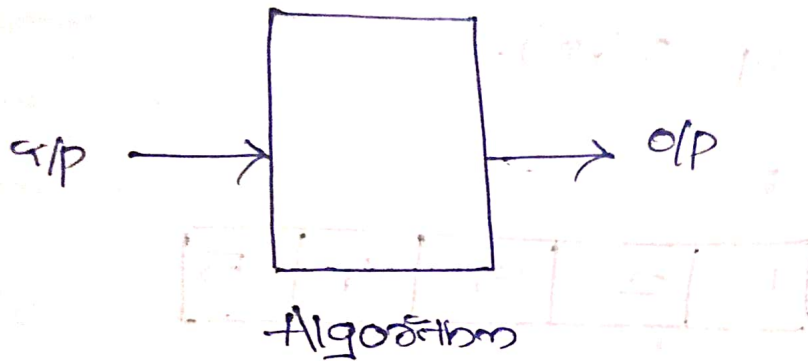
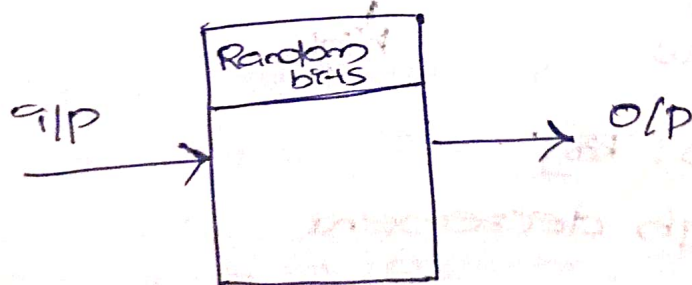


Randomized Algorithm

In Deterministic Algorithm, always gives the same output for the same input and follows a fixed set of steps. It can fall into worst case performance



Randomized algorithm, that uses random numbers to decide what to do next anywhere in its logic. It can produce different performance even when given the same input.



Typically, this randomness is used to reduce time complexity or space complexity. The algorithm works by generating random numbers or randomly making choice on i/p.

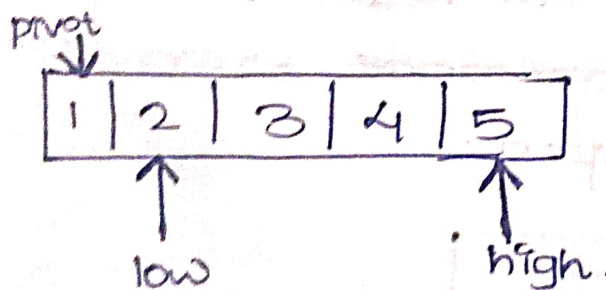
• Randomized Quick Sort algorithm.

In a deterministic quick sort, the time complexity will be increased if on inputs are already in a sorted order. This scenario give worst case time complexity $O(n^2)$.

Example:

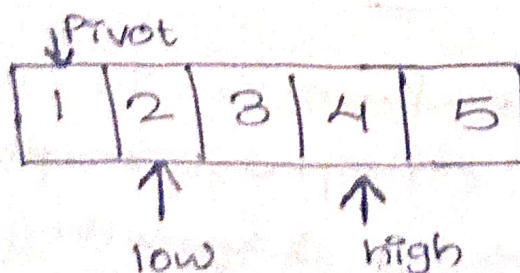
1	2	3	4	5
---	---	---	---	---

(i) select pivot, initialize low pointer & high pointer



ii). $2 > 1$ stop. ~~high~~

• $5 > 1$, high decrement

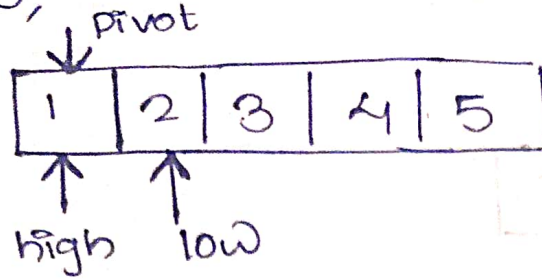


• $4 > 1 \rightarrow$ high decrement

$3 > 1 \rightarrow$ high decrement

$2 > 1 \rightarrow$ high decrement

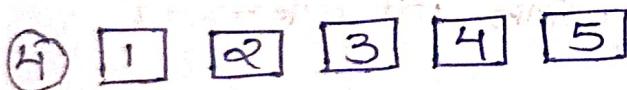
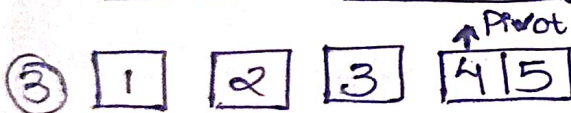
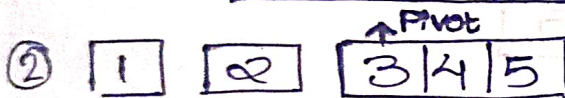
Now,



Whenever low & high crosses then we swap high position with pivot value.

But here, pivot & high pointing to same element. Now pivot element is sorted. So dividing the array into 2 parts.

• After completing ~~one~~ partition.



This problem requires $n-1$ partitions. It leads to worst case time complexity $O(n^2)$.

To avoid such a problem, randomized quick sort has been introduced.

Example:

1	2	3	4	5
---	---	---	---	---

In this method the pivot element is randomly chosen, and interchange the pivot with first element in an array then proceeds the normal quick sort on it.

(i) Let pivot = 3

1	2	3	4	5
---	---	---	---	---

↑
pivot

(ii) interchange pivot with 1st element

3	2	1	4	5
---	---	---	---	---

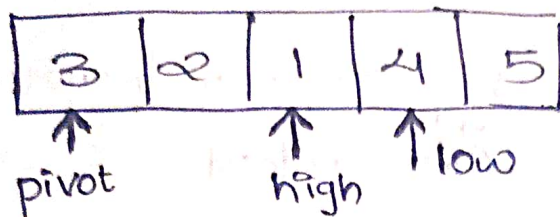
↑ ↑ ↑
pivot low high

(iii) $2 < 3$, low increment.
 $1 < 3$, low increment.
 $4 \neq 3$, stop.

3	2	1	4	5
---	---	---	---	---

↑ ↑ ↑
pivot low high

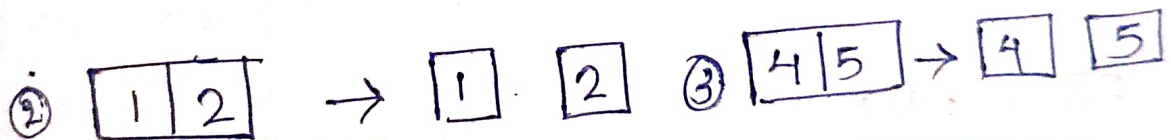
- (iv) 5 < 3, high decrement
 4 < 3, high decrement.
 1 < 3, stop.



low & high crosses, then swap high index value and pivot element.



Again pick random pivot in left as well as right subarray



1. if $low < high$ then
2. $pivot_index \leftarrow RANDOM(low, high)$ // Pick a random index
3. swap $A[pivot_index]$ with $A[low]$ // Move pivot to the start
4. $p \leftarrow PARTITION(A, low, high)$ // Partition the array
5. $RANDOMIZED-QUICKSORT(A, low, p - 1)$ // Recur on left part
6. $RANDOMIZED-QUICKSORT(A, p + 1, high)$ // Recur on right part

```

1. pivot  $\leftarrow A[\text{low}]$  // First element is the pivot
2. left  $\leftarrow \text{low} + 1$ 
3. right  $\leftarrow \text{high}$ 
4. while true do
5.   while left  $\leq$  right and  $A[\text{left}] \leq \text{pivot}$  do
6.     left  $\leftarrow \text{left} + 1$ 
7.   while  $A[\text{right}] \geq \text{pivot}$  and right  $\geq$  left do
8.     right  $\leftarrow \text{right} - 1$ 
9.   if right < left then
10.    break
11.  else
12.    swap  $A[\text{left}]$  with  $A[\text{right}]$ 
13. swap  $A[\text{low}]$  with  $A[\text{right}]$  // Place pivot in correct position
14. return right // Return pivot's sorted index

```

Types of Randomized algorithm

i) Las Vegas Algorithm

- output is always correct & optimal.
- Running time is ^{not} bounded
- correctness is guaranteed, not the time.

ii) Monte Carlo Algorithm

- May produce correct output with some probability.
- This algorithm runs for a fixed no. of steps. It stops when time is up.

Example

Aim: Finding an 'a' in array of n elements

Input: An array of $n \geq 2$ elements,
in which half are 'a's & the other half are 'b's.

Output: find 'a' in an array.

* Las Vegas Algorithm

Algorithm findingA-LV(A, n)

```
{
  repeat
  {
    Randomly choose one element out of
    n elements
  } until ('a' is found)
}
```

This algorithm always give the correct output. They use randomness in their steps, so the time taken may vary, but the algorithm will always terminate & give the right result.

This algorithm succeeds with probability $1/2$.

The expected no. of trials = 2.

$\therefore$ Time complexity = $O(1)$

* Monte Carlo Algorithm

Algorithm findingA-MC(A, n, k)

```
{  
  i = 0;  
  repeat  
  {  
    Randomly select one element out  
    of n elements  
    i = i + 1;  
  } until (i = k or 'a' is found);  
}
```

This algorithm doesn't guarantee success, but the run time is bounded.

The no. of iteration is always less than k

$\therefore$ Time complexity = $O(k)$